QuillAudits

AUDIT REPORT

October 2026

For



Table of Content

Executive Summary 03

Number of Security Issues per Severity 05

Summary of Issues 06

Checked Vulnerabilities 07

Techniques and Methods 09

Types of Severity 11

Types of Issues 12

Severity Matrix 13

High Severity Issues 14

1. Used certificates come back reusable, so the same gold can be sold twice 14

2. Switching to a new vault traps every gold bar and freezes all outstanding TROY 17

Medium Severity Issues 20

3. Re-approving a delayed withdrawal gives the custodian one hour instead of seven days 20

4. A price safety switch ships turned off and nothing in the deployment turns it on 23

Low Severity Issues 25

5. A shipped migration script can never succeed 25

Functional Tests 27

Fuzzing Tests 27

Formal Verification Tests 29

Threat Model 34

1. External Dependencies & Trust Boundaries 34

2. Entry & Exit Points Analysis (Function-Level) 37

3. Asset Flow Mapping (Critical Contracts) 57

Automated Tests 62

Closing Summary & Disclaimer 63

Executive Summary

Project Name	Aum Money
Protocol Type	Gold-backed RWA / Tokenized Gold Protocol
Project URL	https://aum.money/
Overview	AUM is a gold-backed token protocol where serialized physical gold bars are represented by certificate NFTs. Approved certificates are deposited into VaultManager and converted into an equivalent amount of TROY. Exchange provides reserve-backed TROY/USDC and TROY/USDT swaps using gold/USD and stablecoin/USD price feeds.
Audit Scope	The scope of this Audit was to analyze the Aum Money Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/AUMFintech/contracts/tree/fix/2-role-collision-comments

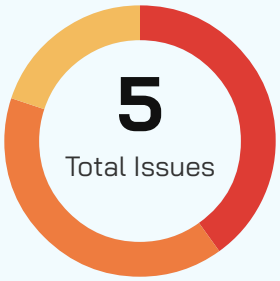
Contracts in Scope	`src/Exchange.sol`
	`src/TroyToken.sol`
	`src/CertificatesNFT.sol`
	`src/ComplianceRegistry.sol`
	`src/VaultManager.sol`
	`src/PriceOracle.sol`
	`src/interfaces/ISanctionsList.sol`
	`src/interfaces/ITroyToken.sol`
	`src/interfaces/ICertificatesNFT.sol`
	`src/interfaces/IPriceOracle.sol`
	`src/interfaces/IComplianceRegistry.sol`
	`src/libs/VaultValidation.sol`
	`src/libs/Roles.sol`
	`src/libs/VaultTypes.sol`
	`src/libs/AddressValidation.sol`
	`src/libs/Decimals.sol`

Branch	fix-2-role-collision-comments
Language	Solidity
Blockchain	EVM-compatible blockchain

Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	18th August 2026 - 27th August 2026
Updated Code Received	18th October 2026
Review 2	19th October 2026
Fixed In	https://github.com/AUMFintech/contracts/commit/2d732cd5e31351a8cbb939f01f4eaf87dcef39ba

✅ Verify the Authenticity of Report on QuillAudits Leaderboard:
https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0%)
High	2 (40%)
Medium	2 (40%)
Low	1 (20%)
Informational	0 (0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	**1**	**1**	0
Partially Resolved	0	0	0	0	0
Resolved	0	**2**	**1**	0	0

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Used certificates come back reusable, so the same gold can be sold twice	High	Resolved
2	Switching to a new vault traps every gold bar and freezes all outstanding TROY	High	Resolved
3	Re-approving a delayed withdrawal gives the custodian one hour instead of seven days	Medium	Acknowledged
4	A price safety switch ships turned off and nothing in the deployment turns it on	Medium	Resolved
5	A shipped migration script can never succeed	Low	Acknowledged

Checked Vulnerabilities

- [x] Access Management
- [x] Arbitrary write to storage
- [x] Centralization of control
- [x] Ether theft
- [x] Improper or missing events
- [x] Logical issues and flaws
- [x] Arithmetic Computations Correctness
- [x] Race conditions/front running
- [x] SWC Registry
- [x] Re-entrancy
- [x] Timestamp Dependence
- [x] Gas Limit and Loops
- [x] Exception Disorder
- [x] Gasless Send
- [x] Use of tx.origin
- [x] Malicious libraries
- [x] Compiler version not fixed
- [x] Address hardcoded
- [x] Divide before multiply
- [x] Integer overflow/underflow
- [x] ERC's conformance
- [x] Dangerous strict equalities
- [x] Tautology or contradiction
- [x] Return values of low-level calls
- [x] Missing Zero Address Validation
- [x] Private modifier
- [x] Revert/require functions
- [x] Multiple Sends
- [x] Using suicide
- [x] Using delegatecall

- [x] Upgradeable safety
- [x] Using throw
- [x] Using inline assembly
- [x] Style guide violation
- [x] Unsafe type inference
- [x] Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

1. Used certificates come back reusable, so the same gold can be sold twice

Resolved

Path	`src/VaultManager.sol:682-684`
Function Name	`completeWithdraw`

Description

When a customer takes their physical gold out of the vault, the system hands the certificate back to them in perfect condition. Nothing marks it as used.

So a customer can walk out with the gold bar in one hand and a fully valid title deed for it in the other, then hand that title deed straight back in and get a fresh batch of TROY for gold that is no longer there.

The tool to prevent this already exists. `CertificatesNFT.burn` destroys a certificate, only the vault can call it, and it requires the vault to be holding the certificate, which is exactly the situation at the moment of release. It is never called anywhere in `src/`.

Two things make it worse:

- Issuing a new certificate needs a certifier plus a one time use custody reference proving the bar arrived. Re-depositing an old certificate needs none of that.
- Because certificates are never burned, the bar's serial number stays reserved forever, so a bar that genuinely comes back can never get a fresh properly checked certificate. Reusing the stale one is the only way back in, which makes the dangerous action the routine one.

Impact

Total loss of the vault's gold.

TROY is a shared claim on a shared pool, and the operator releases whichever bar it likes. So this is not a one time double count. One certificate can be recycled again and again to walk out with the entire vault. Measured: a customer puts in **one bar** and leaves with **four**, the vault ends up **completely empty**, and three honest customers hold tokens backed by nothing.

Throughout, the protocol's own health check `onChainBackingMatchesSupply()` reports everything is fine, because the fake backing is added to both sides of the ledger at once.

Likelihood

Medium. An operator has to approve each deposit, but the operator has nothing to go on:

1. Used certificates come back reusable, so the same gold can be sold twice *(continued)*

- On chain, a used certificate is identical to a fresh one.
- There is no way to ask "has this certificate been redeemed before?". Finding out means manually digging through every past withdrawal.
- Approving deposits is not the physical custody check. The docs give that job to the certifier, who is never consulted on a re-deposit.
- Replays are blocked everywhere else in the system (custody references and shipment references are both one time use), so relying on an operator's memory here is out of step with the rest.

No insider malice needed. An ordinary customer plus an operator doing their documented job is enough.

POC

```
function test_POC_oneBarCertificateDrainsTheWholePool() public {
  address[3] memory victims = [bob, address(0xCA401), address(0xDA7E)];

  // Alice puts exactly ONE bar into the system and cycles it out once.
  uint256 aliceCert = _issueCertificate(alice, "BAR-ALICE", keccak256("CUSTODY-ALICE"));
  _deposit(alice, aliceCert);
  barsInVault += 1;                              // her bar physically arrives
  _withdraw(alice, TEN_OZ_1E18, aliceCert, keccak256("FULFILL-ALICE"));
  barsInVault -= 1;                              // and physically leaves again

  // Three honest depositors each place a genuine bar.
  for (uint256 i = 0; i < victims.length; i++) {
    uint256 cert = _issueCertificate(
      victims[i], string.concat("BAR-V", vm.toString(i)), keccak256(abi.encode("CUSTODY-V", i))
    );
    _deposit(victims[i], cert);
    barsInVault += 1;
  }
  assertEq(barsInVault, 3, "three real bars in the vault");

  // Alice recycles the certificate she already redeemed, once per victim.
  uint256 heldCert = aliceCert;
  uint256 aliceBars = 1;                         // she already has her own bar back
  for (uint256 i = 0; i < victims.length; i++) {
    _deposit(alice, heldCert);                   // phantom backing: no gold arrives
    uint256 victimCert = certs.tokenIdBySerial(string.concat("BAR-V", vm.toString(i)));
    _withdraw(alice, TEN_OZ_1E18, victimCert, keccak256(abi.encode("FULFILL-DRAIN", i)));
    barsInVault -= 1;                            // a real bar ships to Alice
    aliceBars += 1;
    heldCert = victimCert;                       // handed back, ready for next round
  }

  // HARM
  assertEq(aliceBars, 4, "Alice holds 4 physical bars after contributing 1");
  assertEq(barsInVault, 0, "the vault is physically empty");
  assertEq(troy.totalSupply(), 3 * TEN_OZ_1E18, "30 TROY outstanding against nothing");
```

1. Used certificates come back reusable, so the same gold can be sold twice *(continued)*

```
  assertEq(vault.totalBackedOzMicro(), 3 * TEN_OZ_MICRO, "ledger claims 30 oz of backing");
  assertTrue(vault.onChainBackingMatchesSupply(), "solvency view true with an empty vault");
}
```

Recommendation

Option A, destroy the certificate on release.

```
  certificatePositions[id] =
-     VaultTypes.CertificatePosition({state: VaultTypes.CertificateState.OutsideVault,
activeRequestId: 0});
- nft.transferFrom(me, recipient, id);
+     VaultTypes.CertificatePosition({state: VaultTypes.CertificateState.None, activeRequestId:
0});
+ nft.burn(id);
```

Verified: this kills all three exploits. It also frees the serial number so a returning bar can be readmitted properly, with a certifier and a fresh custody proof, which is what the docs say burning is for. Note that 7 existing tests assert the certificate comes back to the customer, so returning it is deliberate and this is a product decision, not just a code change.

Option B, mark it spent (recommended). Add a `Redeemed` state and park the certificate there instead of returning it to the depositable state. Deposits then reject it automatically. Add a certifier only function that moves it back to usable, but only when a new one time custody proof is supplied, the same standard as a first deposit. The customer keeps the certificate as a document travelling with their bar, it just cannot create money until someone confirms the bar is back.

Either way, add a test asserting a returned certificate cannot mint TROY again, and expose a way to check on chain whether a certificate has been redeemed.

Status

Resolved

High Severity Issues

2. Switching to a new vault traps every gold bar and freezes all outstanding TROY

Resolved

Path	`src/TroyToken.sol:59-65` `src/CertificatesNFT.sol:111-117`
Function Name	`setVault`

Description

The certificate contract and the TROY token each store a pointer to "the vault", which is the only address allowed to create or destroy tokens and certificates. The owner can change that pointer at any time. The only checks are that the new address is not empty and is a contract. Nothing verifies the old vault has been emptied first.

The moment the pointer moves, the old vault becomes a locked box:

- Releasing a stored bar requires destroying the customer's TROY, and that is restricted to whichever vault the pointer currently names. The old vault is no longer that vault, so it fails.
- The emergency "rescue a stuck NFT" function deliberately refuses to touch any certificate that is backing tokens, so there is no back door.
- The new vault starts blank. It has no record of the existing bars and there is no function to teach it about them. The certificates are also still owned by the old vault, which cannot release them.

There is a second, quieter effect. Both assets have an emergency freeze on ordinary transfers, with an exemption so the vault can still unwind pending customer requests. That exemption follows the pointer. The instant it moves, the old vault, still holding customer money, loses its exemption.

Both shipped migration scripts do this pointer change with no safety check (`script/WireProtocol.s.sol:26-31`, `script/DeployAndWireVaultSingleAdmin.s.sol:93-94`).

Impact

Every gram of gold in the vault becomes unrecoverable, and every TROY token in circulation becomes permanently unredeemable. There is no admin override and no upgrade that fixes it, because the certificates are stuck in a contract that has lost its authority to move them.

This is unrecoverable by design. The rescue function's refusal to touch backing certificates rests on the assumption that a normal withdrawal always exists as an exit, and that assumption is exactly what breaks here.

2. Switching to a new vault traps every gold bar and freezes all outstanding TROY *(continued)*

Likelihood

Medium. Nobody has to act maliciously. This is what happens when an operator follows the project's own migration scripts on a live system. It needs the owner key, which is a trusted multisig, but the trigger is a routine documented maintenance operation, not an abuse of privilege

POC

```
function test_POC_F2_repointStrandsBackingAndBricksTroy() public {
  uint256 bar = _issueAndDeposit(alice, "BAR-A", keccak256("CUSTODY-A")); // 1 bar in, 10 TROY
out

  // The setter validates only non-zero + code.length > 0. No drained check.
  VaultManager successor = _deployVault();
  vm.startPrank(admin);
  troy.setVault(address(successor));
  certs.setVault(address(successor));
  vm.stopPrank();

  // 1. The only exit for a stored bar now fails inside TroyToken.burn.
  vm.startPrank(alice);
  troy.approve(address(vault), type(uint256).max);
  uint256 wReq = vault.requestWithdraw(TEN_OZ_1E18); // still works, escrow moves fine
  vm.stopPrank();
  uint256[] memory one = new uint256[](1);
  one[0] = bar;
  vm.prank(operator);
  vault.approveWithdraw(wReq, one);                  // still works, pure bookkeeping

  vm.prank(custodian);
  vm.expectRevert(TroyToken.NotVault.selector);
  vault.completeWithdraw(wReq, keccak256("FULFILL-A")); // HARM: redemption is dead

  // 2. rescueERC721 refuses the certificate while reserved AND after unwinding to Backing.
  // (two expectRevert blocks, see test file)

  // 3. The successor cannot adopt the certificate.
  (VaultTypes.CertificateState succState,) = successor.certificatePositions(bar);
  assertEq(uint8(succState), uint8(VaultTypes.CertificateState.None), "successor never heard of
it");
  assertEq(successor.totalBackedOzMicro(), 0,       "successor starts with zero backing");
  assertEq(certs.ownerOf(bar), address(vault),      "NFT still trapped in the old vault");

  // 4. Alice's TROY is unredeemable anywhere.
  vm.startPrank(alice);
  troy.approve(address(successor), type(uint256).max);
  uint256 succReq = successor.requestWithdraw(TEN_OZ_1E18);
  vm.stopPrank();
  vm.prank(operator);
  vm.expectRevert(VaultManager.NotOwner.selector);
  successor.approveWithdraw(succReq, one);           // HARM: no redemption path exists
```

2. Switching to a new vault traps every gold bar and freezes all outstanding TROY *(continued)*

```
  assertEq(troy.totalSupply(), TEN_OZ_1E18, "10 TROY outstanding and permanently unbacked");
}
```

```
[PASS] test_POC_F2_repointStrandsBackingAndBricksTroy()
[PASS] test_POC_F2b_repointRevokesTheOldVaultsPauseExemption()
```

The second test shows the freeze effect: with the emergency freeze on, a refund from the old vault works before the pointer moves and reverts `TransfersPaused` after, while that vault still holds customer money.

Recommendation

Do both of the following. Either one alone leaves you stuck.

Block the unsafe switch.

```
  function setVault(address vault_) external onlyOwner {
      if (vault_ == address(0)) revert ZeroAddr();
      AddressValidation.requireContract(vault_);
+     if (vault != address(0)) {
+         if (IVaultManager(vault).totalBackedOzMicro() != 0
+             || IVaultManager(vault).totalLockedTroy() != 0
+             || IVaultManager(vault).pendingRequests() != 0) revert VaultNotDrained();
+     }
      address oldVault = vault;
```

Add a proper exit. The check above on its own means a vault holding any gold can never be replaced, which is the same trap by another name. Add an admin only, paused mode function that releases a stored certificate for migration and reduces the recorded backing to match.

Warning: the obvious version of that function moves the certificate to the ordinary depositable state, which walks straight into Finding 1. Send it to a terminal spent state instead.

Also consider making the freeze exemption cover recent vaults, not just the current one, so an outgoing vault can still refund customers during a migration.

Status Resolved

Medium Severity Issues

3. Re-approving a delayed withdrawal gives the custodian one hour instead of seven days

Acknowledged

Path	src/VaultManager.sol

Description

When an operator approves a gold withdrawal, a seven day clock starts. If the custodian has not shipped within that window, the customer may cancel and take their tokens back.

Operators can also revoke an approval, which is the documented way to handle a shipping delay and re-assign later. Revoking resets the status and clears several fields, but it does not reset the clock. When the request is approved again, the original seven day deadline is still running.

So an operator can re-approve on day 6 and what looks like a fresh seven day window is actually one hour. The custodian sees a new approval and has no reason to suspect otherwise.

There is a second problem in the same place, not in the original report. If the request is left revoked past the seven day mark it can never be approved again, because the system rejects it as expired. An operator using the documented delay procedure can permanently destroy a customer's ability to receive their gold. Only cancel or reject remain.

Impact

The customer ends up with both the gold and the money. Once the stale deadline passes they cancel and get a 100% refund, while the bar has already shipped. The books then count that bar as backing again, so the shortfall is invisible and the health check reports full solvency.

This is the same end state as Finding 1, reached through a different door.

The second problem is a service failure rather than a loss: a withdrawal becomes permanently unfulfillable and must be torn down and restarted.

Likelihood

Medium. It needs an operator to use revoke and re-approve late in the window, which is exactly what that flow exists for, since delayed international bullion shipments are the reason it is there.

The customer plans nothing. They cancel when the system allows it, which looks completely legitimate from their side. The custodian gets caught out and has no on chain signal that the window is closing.

The docs acknowledge that cancel and complete can race after the deadline. What they do not acknowledge, and what makes this a finding, is that revoke and re-approve silently compresses the window without telling anyone.

3. Re-approving a delayed withdrawal gives the custodian one hour instead of seven days *(continued)*

POC

```
function test_POC_F5_reapprovalInheritsStaleDeadline() public {
  uint256 bar = _issueAndDeposit(alice, "BAR-A", keccak256("CUSTODY-A"));
  vm.startPrank(alice);
  troy.approve(address(vault), type(uint256).max);
  uint256 wReq = vault.requestWithdraw(TEN_OZ_1E18);
  vm.stopPrank();

  uint256[] memory one = new uint256[](1);
  one[0] = bar;

  uint256 t0 = block.timestamp;
  vm.prank(operator);
  vault.approveWithdraw(wReq, one);
  anchor = vault.getWithdrawalRequest(wReq).approvalTimestamp;
  assertEq(uint256(anchor), t0, "first approval anchors the deadline at t0");

  // Operator revokes. The deadline anchor survives the reopen.
  vm.prank(operator);
  vault.revokeWithdrawApproval(wReq, "logistics delay");
  assertEq(vault.getWithdrawalRequest(wReq).approvalTimestamp, anchor,
    "revoke clears status but NOT approvalTimestamp");

  // Re-approved one hour before the ORIGINAL deadline.
  vm.warp(t0 + 7 days - 1 hours);
  vm.prank(operator);
  vault.approveWithdraw(wReq, one);
  VaultTypes.WithdrawalRequest memory r = vault.getWithdrawalRequest(wReq);
  assertEq(r.approvalTimestamp, anchor, "HARM: fresh approval carries a 7-day-old deadline");
  assertEq(uint256(r.approvalTimestamp) + 7 days, block.timestamp + 1 hours,
    "HARM: the just-granted approval expires in one hour");

  // The custodian ships the bar. One hour later the customer cancels and takes the money back.
  vm.warp(t0 + 7 days + 1);
  vm.prank(alice);
  vault.cancelWithdraw(wReq);

  assertEq(troy.balanceOf(alice), TEN_OZ_1E18, "HARM: full refund after the bar shipped");
  (VaultTypes.CertificateState st,) = vault.certificatePositions(bar);
  assertEq(uint8(st), uint8(VaultTypes.CertificateState.Backing),
    "HARM: shipped bar counted as backing again");
  assertTrue(vault.onChainBackingMatchesSupply(), "protocol reports itself solvent");
}
```

```
[PASS] test_POC_F5_reapprovalInheritsStaleDeadline()
[PASS] test_POC_F5b_revokeAfterDeadlinePermanentlyBlocksReapproval()
```

The second test approves, revokes, waits past day 7, and shows re-approval is rejected as expired, permanently.

3. Re-approving a delayed withdrawal gives the custodian one hour instead of seven days *(continued)*

Recommendation

One line fix. Clear the deadline when a revoked request is reopened, alongside the fields already cleared there:

```
  function _reopenWithdrawalRequest(VaultTypes.WithdrawalRequest storage request) internal {
      request.status = VaultTypes.WithdrawalRequestStatus.Pending;
      request.approvedBy = address(0);
      request.completionTimestamp = 0;
+     request.approvalTimestamp = 0;
      pendingRequests++;
  }
```

Each new approval then gets its own full seven days and the permanent expiry problem disappears too.

If keeping the original deadline is genuinely intended, make it visible: return the remaining time from the approval function, put it in the event, and reject any re-approval that would leave less than a minimum settlement window. A custodian should never accept a request without knowing how long they have.

Status

Acknowledged

Team's Comment

Preserving the first approval timestamp is deliberate. The purpose is to prevent an operator from repeatedly revoking and reapproving a withdrawal to extend the customer's escrow lock indefinitely.

Medium Severity Issues

4. A price safety switch ships turned off and nothing in the deployment turns it on

Resolved

Path	`src/Exchange.sol:173-187`
Function Name	`_skewOk`

Description

Every swap combines two price readings: the price of gold, and the price of the stablecoin being traded. Each is separately checked for freshness, and neither may be older than about 65 minutes.

But two readings can both be individually fresh and still come from very different moments. A gold price from right now paired with a stablecoin price from 65 minutes ago passes both checks. The contract has a setting to limit that gap.

That setting is missing from the deployment configuration, is never assigned at startup, and so defaults to zero. Zero means "no limit, skip the check". No deployment script turns it on. The go live checklist script, which verifies oracle health, reserves and compliance before opening trading, never checks it either.

The code's own comment describes what is being left open: combining prices from different moments is "an arbitrage/manipulation surface even when each feed independently passes its own freshness check".

Impact

Traders can extract value from the exchange's reserves whenever either price is moving. The clearest case is a stablecoin losing its peg. If the stablecoin's feed has not updated in the last hour, the exchange still values it at one dollar while the market values it at ninety cents, and an arbitrageur buys gold tokens with the cheap stablecoin at the stale rate.

Measured at the shipped settings, a 100,000 unit swap during a 10% depeg extracts about $9,500, while the trading fee meant to price this risk collects $450. That is roughly 21 times the fee. The loss falls on the exchange's liquidity providers.

Likelihood

Medium. Nothing needs to be hacked and no special access is required. It is an ordinary swap, available to anyone, whenever conditions line up.

What holds it back from higher: the 65 minute freshness limit still caps how stale either price can get, so the size of the opportunity is bounded by how much prices moved in that window. On a quiet day the 0.5% fee absorbs it. It takes a real depeg or a sharp gold move to become clearly profitable, so it is an intermittent opportunity rather than a standing free option.

4. A price safety switch ships turned off and nothing in the deployment turns it on *(continued)*

POC

```
function test_POC_F4_skewGuardShipsDisabledAndPermitsMaxSkewSwap() public {
  assertEq(exchange.maxPriceSkew(), 0, "initialize never assigns maxPriceSkew");

  // A depeg. USDC is really worth $0.90 now, but its feed last printed the peg exactly
  // priceUpdateTimeout ago. The gold feed is current. Both pass their own freshness checks.
  uint256 realUsdcValue = 90e16;                  // $0.90
  goldOracle.set(GOLD_USD, block.timestamp, true);
  usdcOracle.set(PEG_USD, block.timestamp - DEPLOY_TIMEOUT, true);

  (,,,, bool healthy) = exchange.getQuoteStableToTroy(address(usdc), 100_000e6);
  assertTrue(healthy, "HARM: a maximally skewed feed pair quotes as healthy");

  uint256 stableIn = 100_000e6;                   // 100,000 USDC
  vm.prank(attacker);
  uint256 troyOut = exchange.exchangeStableToTroy(address(usdc), stableIn, 0);

  // Paid vs received, both valued at the REAL usdc price.
  uint256 paidUsd18     = (uint256(stableIn) * 1e12 * realUsdcValue) / 1e18;
  uint256 receivedUsd18 = (troyOut * GOLD_USD) / 1e18;
  uint256 profitUsd18   = receivedUsd18 - paidUsd18;
  uint256 feeUsd18      = (paidUsd18 * DEPLOY_FEE_BPS) / 10_000;

  assertGt(receivedUsd18, paidUsd18, "HARM: attacker receives more value than paid");
  assertGt(profitUsd18, feeUsd18 * 10, "HARM: extraction dwarfs the fee");
}
```

The control test proves the safeguard works and just needs switching on. With the limit set to 10 minutes, the identical swap is rejected with `PriceSkew`.

Recommendation

Add the setting to the deployment config and assign it at startup, requiring a non-zero value inside the allowed range:

```
   priceUpdateTimeout = config.priceUpdateTimeout;
+ maxPriceSkew = config.maxPriceSkew;   // add to InitConfig; require non-zero and <=
MAX_PRICE_SKEW_LIMIT
+ emit MaxPriceSkewSet(0, config.maxPriceSkew);
```

Tune it to the slower of the two feeds. Too tight and legitimate trades get rejected.

Also add it as a hard requirement in the go live checklist script, so trading cannot open while the switch is off. That script already gates on oracle health, reserves and compliance, and this belongs in the same list.

Status Resolved

Low Severity Issues

5. A shipped migration script can never succeed

Acknowledged

Path	script/DeployAndWireVaultSingleAdmin.s.sol

Description

The protocol requires its five vault roles (admin, upgrader, operator, certifier, custodian) to be five different addresses. This is enforced when a vault is created, and creation is rejected if any two match.

One of the shipped migration scripts, which describes itself in its own comments as a live production migration, hardcodes all five roles to the same address. The vault it tries to create is therefore always rejected and the script always fails. It has never worked and never will.

Its sibling script does the same thing on purpose and says so plainly in its header, "retained for controlled failure testing". This one carries no such warning.

Its success checks are also incomplete. It confirms the new vault's pointers, dependencies and roles, but never checks whether the old vault still holds gold, tokens or pending requests. So if the role problem were fixed, the script would report success while creating exactly the trapped vault described in Finding 2.

Impact

Wasted operational effort and a misleading file in the repository. An operator preparing a migration follows a documented procedure that cannot work, and only finds out when it fails.

Correction to the original report. It claimed the script pauses the live vault before failing, leaving production frozen with no replacement. That was tested against a real node and **does not happen**. The pause step runs during a local dry run, but the tool only sends transactions after the whole script completes successfully. Since it always fails, nothing is sent and the live system is untouched. This holds with standard settings and with simulation disabled, and the project's own build commands do not change it.

So the impact is a broken tool, not a production outage. Severity is Low, not High.

Likelihood

It fails 100% of the time, but the consequence is only wasted time. The dangerous part, the missing drained vault checks, never runs because the script dies before reaching it.

Recommendation

Read the five role addresses from configuration instead of hardcoding them, and check they are all different before doing anything else:

5. A shipped migration script can never succeed *(continued)*

```
- address admin = ADMIN; address upgrader = ADMIN; address operator = ADMIN;
- address certifier = ADMIN; address custodian = ADMIN;
+ address admin     = vm.envAddress("VAULT_ADMIN");
+ address upgrader  = vm.envAddress("VAULT_UPGRADER");
+ address operator  = vm.envAddress("VAULT_OPERATOR");
+ address certifier = vm.envAddress("VAULT_CERTIFIER");
+ address custodian = vm.envAddress("VAULT_CUSTODIAN");
+ _requireDistinct([admin, upgrader, operator, certifier, custodian]);
+ require(currentVault.totalBackedOzMicro() == 0 && currentVault.pendingRequests() == 0
+     && currentVault.totalLockedTroy() == 0, "outgoing vault not drained");
  vm.startBroadcast();
- currentVault.pause();
+ // deploy and verify the replacement first, then pause
```

Add the drained vault precondition, and see Finding 2, because that check alone can leave you with a vault that can never be replaced.

Reorder so the replacement is deployed and verified before the live system is paused. Even though the current failure mode is harmless, "pause production first, build the replacement second" is the wrong order and will eventually bite.

Finally, either give this script the same explicit "not for deployment" header its sibling carries, or make it actually work. Right now it advertises itself as a live migration path and is not one.

Status Acknowledged

Team's Comment

`DeployAndWireVaultSingleAdmin.s.sol` is a test script and will not be used for the main deployment. Its header already describes a test migration.

Functional Tests

Functional Tests

14 checks

Some of the tests performed are mentioned below:

- Verified successful deployment and initialization of all in-scope contracts.
- Verified role-based access control for privileged functions.
- Verified token minting and burning flows.
- Verified certificate NFT deposit and withdrawal flows.
- Verified TROY token transfer and approval functionality.
- Verified USDT/USDC exchange and settlement flows.
- Verified oracle price retrieval and validation.
- Verified compliance and blacklist checks.
- Verified blacklisted users are prevented from restricted token operations.
- Verified USDT blacklist behavior and confirmed that transfers involving a blacklisted address revert.
- Verified deposit and withdrawal recovery flows.
- Verified invalid inputs and unauthorized calls revert as expected.
- Verified pause/unpause functionality where applicable.
- Verified protocol state transitions across the main user flows.

Fuzzing Tests

26 checks

Solvency and value conservation

- Verified TROY total supply always equals total gold backing (`totalBackedOzMicro` scaled by 1e12) across all deposit and withdrawal sequences.
- Verified total gold backing equals the summed weight of all certificates held as Backing or WithdrawalReserved, with no phantom backing.
- Verified the vault's TROY balance always covers total locked TROY, and per-user locks sum exactly to the tracked total.
- Verified the Exchange's TROY balance always covers recorded reserves, and its stable balance always covers tracked liquidity plus accumulated fees.
- Verified each withdrawal completion fully accounts its escrow: burned redemption plus refunded remainder equals the escrowed TROY, with none stranded or created.

- Verified deposit and withdrawal unwind paths (reject, cancel, revoke, recover) leave TROY supply and gold backing unchanged, so only approval and completion move them.
- Verified exchange fee separation is exact, with net plus fee equal to gross and no wei lost or created.

Certificate and request state machine

- Verified certificate positions and their active request IDs remain a consistent bidirectional binding at all times.
- Verified no certificate can be reserved by two withdrawal requests at once (no double-spend of backing) across revoke and re-approve interleavings.
- Verified a reopened pending withdrawal carries no stale allocated certificates or weight.
- Verified request status transitions follow the legal graph and terminal states (completed, rejected, cancelled, recovered) never change again.
- Verified consumed custody and fulfillment references can never be replayed.
- Verified request IDs are strictly increasing and each is assigned only once.

Limits and circuit breakers

- Verified the daily TROY mint limit is never exceeded within a UTC day.
- Verified per-user and global daily exchange limits are never breached, and global leaky-bucket usage never exceeds capacity.

Rescue safety and fund isolation

- Verified admin rescue can never reduce the vault's TROY below total locked, and can never move a certificate that is backing or reserved.
- Verified the ERC721 rescue path rejects the TROY token, closing the shared `transferFrom` selector collision with the ERC20 path.
- Verified one user's swap or withdrawal can never reduce another user's balance or locked TROY.

Compliance robustness

- Verified `isCompliant` always implies `isTransferAllowed` (the stricter policy implies the looser one) for every account.
- Verified a manually blocked account fails both compliance checks under every sanctions and allowlist configuration.
- Verified compliance checks never revert or consume unbounded gas for any sanctions-oracle behavior (reverting, malformed, codeless, or gas-heavy), so transfers cannot be bricked by a hostile feed.
- Verified recovery flows are restricted to users who are not transfer-allowed, so a compliant user's assets cannot be seized.

Oracle robustness

- Verified the price oracle read never reverts for any feed behavior and returns valid only when the price is fresh, positive, within bounds, and the sequencer (if configured) is up past its grace period.

- Verified no swap executes on a stale, invalid, or excessively skewed price.

Precision and emergency handling

- Verified all price and fee conversions round in the protocol's favor, so swap outputs never exceed the ideal value and conversion round-trips never profit the user.
- Verified pause and emergency mode halt swaps and quoting while leaving reserve management (liquidity and fee operations) available.

Formal Verification Tests 58 checks

System-level (cross-contract)

- Proved TROY total supply equals total gold backing (`totalBackedOzMicro` times 1e12) for all reachable states while the token is wired to the vault (`inv_system_backingMatchesSupplyWhileWired`).
- Proved total gold backing equals the summed weight of every certificate in a backing-counted state (`inv_system_backingIsSumOfWeights`).
- Proved no path mints TROY without corresponding backing, and no path drops backing without a burn (`rule_anti_noMintWithoutBacking` , `rule_anti_noBackingDropWithoutBurn` , `rule_system_noWriteDownPathWithoutBurn`).
- Proved approveDeposit and completeWithdraw conserve value jointly across the vault and token contracts (`rule_system_approveDepositConservesAcrossContracts` , `rule_system_completeWithdrawConservesAcrossContracts`).
- Proved a certificate approval mints exactly the attested weight, and a redeemed certificate can never re-back new TROY (`rule_vault_approveCertificateMintsExactly` , `rule_vault_redeemedCertificateCannotRebackTroy`).
- Proved a user is never left ahead in value after a redeem round-trip (`rule_roundtrip_userNeverAheadAfterRedeem`).
- Proved backing always requires a fresh custody attestation, and genesis correctly admits pre-existing supply (`rule_anti_backingRequiresFreshAttestation` , `rule_system_genesisAdmitsPreExistingSupply`).
- Proved the Exchange never changes TROY total supply under any method (`rule_system_exchangeNeverChangesSupply`).
- Proved re-pointing the token's vault cannot strand backing and revokes the old vault's burn authority (`rule_system_repointCannotStrandBacking` , `rule_system_repointRevokesVaultBurnAuthority`).
- Proved a paused protocol never increases value, yet an approved withdrawal still unwinds while paused (`rule_anti_pausedNeverIncreasesValue` , `rule_anti_pausedCompleteWithdrawStillUnwinds`).
- Proved cancellation never strands a shipped bar (`rule_system_cancelNeverStrandsShippedBar`).

VaultManager

- Proved per-user locks always sum to `totalLockedTroy` (`inv_vault_lockedSum`).

- Proved `totalLockedTroy` and `totalBackedOzMicro` change only through their permitted functions (`rule_vault_onlyAllowedChange_totalLocked` , `rule_vault_onlyAllowedChange_totalBacked`).
- Proved consumed custody and fulfillment references are never released (`rule_vault_referencesNeverReleased`).
- Proved each withdrawal escrow is released exactly once, and every allocation stays within its escrow (`rule_vault_escrowReleasedExactlyOnce` , `inv_vault_allocationWithinEscrow`).
- Proved rescue cannot touch escrowed funds, the ERC721 rescue path rejects TROY, and the vault holds no self-allowance over TROY (`rule_vault_rescueCannotTouchEscrow` , `rule_vault_rescueERC721Guarded` , `inv_vault_noSelfAllowanceOverTroy`).
- Proved one actor's operations never reduce another user's lock (`rule_vault_thirdPartyLockIsolation`).
- Proved operator, custodian, certifier, admin, and cancel-owner gating on every privileged function, that the default admin is never grantable, and that roles stay disjoint (`rule_vault_operatorOnly` , `rule_vault_custodianOnly` , `rule_vault_certifierOnly` , `rule_vault_adminOnly` , `rule_vault_cancelOwnerOnly` , `rule_defaultAdminNotGrantable` , `rule_vault_rolesStayDisjoint`).
- Proved recovery is only reachable for non-transfer-allowed users and does not depend on the sanctions oracle being reachable (`rule_vault_recoveryRequiresNonTransferable` , `rule_vault_recoveryIndependentOfSanctionsOracle`).
- Proved no self-attested mint is possible and each attestation is bounded by its request (`rule_anti_noSelfAttestedMint` , `rule_anti_attestationBoundedByRequest`).
- Proved minting never exceeds the daily limit, the limit setter cannot retroactively bypass the day's cap, and the mint-day marker is monotone (`rule_vault_mintNeverExceedsDailyLimit` , `rule_vault_mintLimitSetterNoRegression` , `rule_vault_lastMintDayMonotone`).
- Proved request IDs strictly increment and stay disjoint, the pending counter matches stored requests, and pending count deltas are bounded per call (`rule_vault_requestIdStrictlyIncrements` , `inv_vault_requestIdDisjoint` , `inv_vault_pendingCount` , `rule_vault_pendingDeltaBounded`).
- Proved the withdrawal, certificate-request, deposit-request, and certificate-position state graphs only allow legal transitions, with correct status-timestamp coupling, write-once approval timestamps, and cancel-approved-only-after-expiry (`rule_vault_withdrawalGraph` , `rule_vault_certRequestGraph` , `rule_vault_depositRequestGraph` , `rule_vault_certificateStateGraph` , `inv_vault_positionShape` , `inv_vault_statusTimestampCoupling` , `rule_vault_approvalTimestampWriteOnce` , `rule_vault_cancelApprovedOnlyAfterExpiry`).
- Proved cancellation never strands shipped backing, an approved request always leaves a usable action window, and escrow below the smallest bar is still exitable (`rule_vault_cancelDoesNotStrandShippedBacking` , `rule_vault_approvedRequestLeavesUsableWindow` , `rule_vault_escrowBelowSmallestBarIsStillExitable`).
- Proved the ERC721 receiver rejects untracked certificate transfers (`rule_vault_receiverRejectsUntracked`).
- Proved initialize is one-shot and rejects role collisions, dependencies are immutable after init, and only named setters write pointers (`rule_initializeIsOneShot` , `rule_initRejectsRoleCollision` , `rule_vault_depsImmutableAfterInit` , `rule_pointer_onlyNamedSetterWrites`).

- Proved pause gates the intended functions, with completeWithdraw exempt so redemptions cannot be trapped (`rule_pausePartitionExact` , `rule_pausePartitionExact_completeWithdrawIsExempt`).
- Proved upgrade authorization is exactly the upgrader role (`rule_upgradeAuthorityExact`).

TroyToken

- Proved supply changes only via the vault, mint raises supply exactly by the amount, burn lowers it exactly, mint and burn are inverse, and mint amount is the micro-oz scaling (`rule_token_supplyOnlyChangesViaVault` , `rule_token_mintRaisesSupplyExactly` , `rule_token_burnLowersSupplyExactly` , `rule_token_mintBurnSupplyInverse` , `rule_token_mintAmountIsMicroOzScaled`).
- Proved total supply always equals the sum of balances (`inv_token_supplyIsSumOfBalances`).
- Proved burn never touches a third party's balance, and zero-amount or zero-recipient mint/burn is rejected (`rule_token_burnNeverTouchesThirdParty` , `rule_token_zeroAmountMintBurnRejected` , `rule_token_mintRejectsZeroRecipient`).
- Proved pause blocks peer transfers but exempts vault legs and never blocks mint (`rule_token_pausePartitionExact` , `rule_token_pauseExemptsVaultLegs` , `rule_token_pauseNeverBlocksMint`).
- Proved blocked senders and receivers cannot transfer, vault legs are never screened, and a zero compliance address disables the gate as designed (`rule_token_blockedCannotTransfer` , `rule_token_blockedSenderCannotTransfer` , `rule_token_vaultLegIsNeverScreened` , `rule_token_zeroComplianceDisablesGate`).
- Proved ownership renunciation always reverts, a pending owner has no authority, vault setter rejects zero and keeps a non-zero vault, and re-point revokes the old vault's unwind and compliance carve-outs (`rule_token_renounceAlwaysReverts` , `rule_token_pendingOwnerHasNoAuthority` , `rule_token_setVaultRejectsZero` , `rule_token_setVaultLeavesNonZeroVault` , `rule_token_repointRevokesEscrowUnwindExemption` , `rule_token_repointRevokesVaultComplianceCarveOut`).
- Proved permit only ever changes allowance (`rule_token_permitTouchesOnlyAllowance`).

CertificatesNFT

- Proved mint and burn are vault-only, a non-vault caller cannot mint, and burn requires vault custody (`rule_asset_mintBurnVaultOnly` , `rule_asset_nonVaultCannotMint` , `rule_asset_burnRequiresVaultCustody`).
- Proved token IDs strictly increment, are never zero once initialized, follow only legal lifecycle transitions, and a retired ID is never reissued (`rule_nft_tokenIdStrictlyIncrements` , `rule_nft_tokenIdNeverZeroOnceInitialized` , `rule_nft_lifecycleTransitions` , `rule_nft_retiredIdNeverReissued`).
- Proved the serial index and info index always match live supply, weight is write-once and never zero for a live token, and info and serial index are written only by mint/burn (`inv_nft_serialIndexMatchesSupply` , `inv_nft_infoIndexMatchesSupply` , `rule_nft_weightWriteOnce` , `rule_nft_liveWeightNeverZero` , `rule_nft_infoOnlyMintBurnWrites` , `rule_nft_serialIndexOnlyMintBurnWrites`).

- Proved mint records the attested weight, never overwrites an occupied serial index, burn frees serial control, and a receiver callback cannot retire a certificate mid-mint (`rule_nft_mintRecordsAttestedWeight` , `rule_nft_mintNeverOverwritesSerialIndex` , `rule_nft_burnFreesSerialControl` , `rule_nft_callbackCannotRetireCertificate`).
- Proved pause blocks peer transfers but exempts mint, blocked accounts cannot transfer, and the vault is never screened (`rule_pausePartitionExact` , `rule_nft_mintBypassesPause` , `rule_asset_blockedCannotTransfer` , `rule_asset_vaultIsNeverScreened`).
- Proved owner-gated functions are owner-only, renunciation brakes are in place, and upgrade authority is exact (`rule_asset_ownerGatedOnly` , `rule_asset_ownershipBrakes` , `rule_upgradeAuthorityExact`).

ComplianceRegistry

- Proved the zero account is never allowed, a blocked account is always denied, and an expired allowlist denies protocol entry (`rule_registry_zeroAccountNeverAllowed` , `rule_registry_blockedAccountAlwaysDenied` , `rule_registry_expiredAllowlistDeniesProtocolEntry`).
- Proved the gate is a pure function of state, the policy algebra is total with `isCompliant` implying `isTransferAllowed` , and the transfer policy ignores allowlist expiry by design (`rule_registry_gateIsPureFunctionOfState` , `rule_registry_policyAlgebraAndTotality` , `rule_registry_transferPolicyIgnoresAllowlist`).
- Proved sanctions fail-open approves an unscreened account, fail-closed denies a clean account, an oracle outage denies a clean account, and an absent oracle answer resolves to exactly the policy flag (`rule_registry_failOpenApprovesUnscreenedAccount` , `rule_registry_failClosedDeniesCleanAccount` , `rule_registry_sanctionsOutageDeniesCleanAccount` , `rule_registry_absentOracleAnswerIsExactlyThePolicyFlag`).
- Proved sanctions configuration stays coherent, block and unblock restore prior state, and a block takes effect immediately (`inv_registry_sanctionsCoherent` , `rule_registry_blockUnblockRestores` , `rule_registry_blockTakesEffectImmediately`).
- Proved eligibility and allowlist writers are role-restricted, no eligibility write occurs for the zero account, the role split holds, init rejects role collision, and upgrade authority is exact (`rule_registry_eligibilityWritersRestricted` , `rule_registry_allowlistWritersRestricted` , `rule_registry_noEligibilityWriteForZeroAccount` , `rule_oracleRegistry_roleSplit_eligibility` , `rule_initRejectsRoleCollision` , `rule_upgradeAuthorityExact`).

PriceOracle

- Proved `read()` never reverts, valid implies fresh, valid implies a usable and in-bounds price, a paused read is zero and invalid, and a non-zero price is returned only on a soft failure (`rule_oracle_readNeverReverts` , `rule_oracle_validImpliesFresh` , `rule_oracle_validImpliesUsablePrice` , `rule_oracle_validImpliesWithinBounds` , `rule_oracle_pausedReadIsZeroInvalid` , `rule_oracle_nonZeroPriceOnlyOnSoftFailure`).
- Proved a healthy gold price is reachable, and out-of-band gold or stable prices are rejected (`rule_oracle_healthyGoldPriceIsReachable` , `rule_oracle_goldBandRejectsOutOfBandPrice` , `rule_oracle_stableBandRejectsOutOfBandPrice`).

- Proved setBounds rejects an incoherent band, setStaleAfter rejects out-of-domain values, and parameter, grace-period, and bounds domains stay coherent (`rule_oracle_setBoundsRejectsIncoherentBand` , `rule_oracle_setStaleAfterRejectsOutOfDomain` , `inv_oracle_paramDomains` , `inv_oracle_gracePeriodDomain` , `inv_oracle_boundsCoherent`).
- Proved a configured band is admin-revocable, the freshness window is admin-widenable, and out-of-band or stale prices can recover to healthy after admin action (`rule_oracle_configuredBandIsAdminRevocable` , `rule_oracle_freshnessWindowIsAdminWidenable` , `rule_oracle_outOfBandPriceCanBecomeHealthy` , `rule_oracle_stalePriceCanBecomeHealthyAfterWidening`).
- Proved risk parameters change only via their own setter, the role split holds, the default admin is not grantable, pointers move only via named setters, pause is self-reversible, init rejects role collision, and upgrade authority is exact (`rule_oracle_riskParamsOnlyViaOwnSetter` , `rule_oracleRegistry_roleSplit` , `rule_defaultAdminNotGrantable` , `rule_pointer_onlyNamedSetterWrites` , `rule_oracle_pauseIsSelfReversible` , `rule_initRejectsRoleCollision` , `rule_upgradeAuthorityExact`).

Exchange

- Proved recorded TROY reserves change only through allowed methods and in the correct direction, and fees and principal stay separated (`rule_exchange_onlyAllowedChange_troyReserves` , `rule_exchange_troyReservesDirection` , `rule_exchange_feePrincipalSeparation`).
- Proved TROY and stable solvency are preserved, credit is a measured balance delta, swap deltas are exact, and sweep is bounded by accumulated fees (`rule_exchange_troySolvencyPreserved` , `rule_exchange_stableSolvencyPreserved` , `rule_exchange_creditIsMeasuredDelta` , `rule_exchange_swapDeltasExact` , `rule_exchange_sweepBoundedByFees`).
- Proved the global bucket is never overfilled, per-user volume stays within limit, limit coherence and symmetric daily-cap units hold, and lazy reset and bucket checkpointing are sound (`rule_exchange_globalBucketNeverOverfilled` , `rule_exchange_perUserVolumeWithinLimit` , `rule_exchange_limitCoherence` , `rule_x_dailyCapUnitIsSymmetric` , `rule_exchange_lazyResetSound` , `rule_exchange_bucketCheckpointCoherent`).
- Proved swaps respect skew (including when configured), init leaves skew disabled, a healthy quote may still not execute, no swap runs on an unhealthy price, quote health matches the documented conditions, and no input is consumed for zero output (`rule_exchange_swapRespectsSkew` , `rule_exchange_swapRespectsSkewWhenConfigured` , `rule_exchange_initLeavesSkewDisabled` , `rule_x_healthyQuoteMayNotExecute` , `rule_anti_noSwapOnUnhealthyPrice` , `rule_x_quoteHealthIsExactlyDocumented` , `rule_x_noInputConsumedForZeroOutput`).
- Proved extractable value is bounded by oracle-implied value (`rule_anti_extractionBoundedByOracleValue`).
- Proved admin and operator gating on privileged functions, dependencies are immutable and distinct, removals are gated on the route, pointers move only via named setters, and parameter, timeout, and limit domains hold (`rule_exchange_adminOnly` , `rule_exchange_operatorOnly` , `rule_exchange_depsImmutableAndDistinct` , `rule_exchange_removeGatedOnRoute` , `rule_pointer_onlyNamedSetterWrites` , `inv_exchange_paramDomains` , `inv_exchange_timeoutDomain` , `inv_exchange_limitsNonZero`).
- Proved upgrade authorization is exactly the upgrader role (`rule_exchange_upgradeAuthorityExact`).

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20	Interface	Defines the ERC20 interface used for TROY, USDT, USDC, and other token interactions.	Tokens conform to the expected ERC20 behavior.	Non-standard token behavior may cause transfers or accounting assumptions to fail.
SafeERC20	OpenZeppelin Library	Provides safe wrappers for ERC20 transfers and approvals.	Library correctly handles token return values and failed transfers.	A non-standard or malicious token can still introduce unexpected behavior despite safe wrappers.
AccessControlDefaultAdminRulesUpgradeable	OpenZeppelin Base Contract	Provides role-based access control and delayed default-admin transfers for VaultManager, Exchange, PriceOracle, and ComplianceRegistry.	OpenZeppelin access-control implementation is correct and privileged roles remain trusted.	Compromise or misconfiguration of privileged roles can alter vault, exchange, oracle, or compliance configuration.
UUPSUpgradeable	OpenZeppelin Base Contract	Provides upgradeability for VaultManager, Exchange, PriceOracle, and ComplianceRegistry (upgrader role) and for CertificatesNFT (owner).	The designated upgrader or owner is trusted and upgrade authorization remains secure. Because the TROY and certificate vault bindings are permanent, an in-place upgrade of the VaultManager proxy is the only vault migration path.	A compromised upgrader can replace contract logic and bypass intended controls. A CertificatesNFT upgrade could rebind the certificate vault. Loss of the VaultManager upgrader key removes the ability to repair or migrate the vault.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ComplianceRegistry	Internal Protocol Contract	Provides account-level blocking, allowlist, and sanctions checks used by protocol components.	Consumers correctly reference and enforce the registry's compliance decisions.	Incorrect configuration or disabling of compliance checks can permit restricted accounts to interact.
ISanctionsList / Sanctions Contract	External Contract	ComplianceRegistry queries isSanctioned(address) to determine whether an account is sanctioned.	The configured sanctions source returns correct and canonical sanctions information.	Incorrect, unavailable, malicious, or fail-open sanctions responses can allow sanctioned accounts to interact.
Chainlink Price Feed	External Oracle	Supplies gold/USD and stablecoin/USD prices used by PriceOracle and Exchange for asset valuation and swap calculations.	The configured Chainlink feeds provide accurate, timely, and valid prices.	Stale, incorrect, unavailable, or misconfigured oracle data can result in incorrect pricing or failed operations. Exchange always enforces a non-zero cross-feed timestamp skew below priceUpdateTimeout, but two feeds that are equally stale still pass that check and are bounded only by priceUpdateTimeout.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
TroyToken	Protocol ERC20 Contract	Represents the protocol's gold-backed token and is used by VaultManager and Exchange. The minting vault is fixed at construction.	Token supply and transfer behavior remain consistent with protocol assumptions. Mint and burn authority is permanently bound to the VaultManager proxy and cannot be reassigned; the token is not upgradeable.	Token configuration or transfer restrictions can affect deposits, withdrawals, and swaps. All redemption depends on the VaultManager proxy remaining available and correctly upgradeable, since the token cannot be pointed at a replacement vault.
CertificatesNFT	Protocol ERC721 Contract	Represents serialized physical gold certificates deposited into VaultManager. The vault is bound once at initialization, and certificates are burned when a withdrawal completes.	Certificate ownership and metadata accurately represent the underlying gold entitlement. A live certificate always corresponds to unredeemed gold; a redeemed bar must be re-certified with a fresh custody attestation.	Incorrect certificate state or ownership can affect vault accounting and redemption. The vault binding can only change through an owner-authorized UUPS upgrade, so the owner key is trusted not to rebind it.
VaultSystemDeployer	Internal Deployment Contract	Deploys the VaultManager proxy, TroyToken, and the CertificatesNFT proxy, binds both assets to the vault, and initializes the vault in a single transaction.	The deployer supplies reviewed implementation addresses and correct role configuration; the deployer contract retains no authority after construction.	An unreviewed implementation or wrong configuration produces a misconfigured system. The TROY vault binding cannot be corrected afterwards, so errors there require a fresh deployment or an in-place vault upgrade.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
USDT	External ERC20 Contract	Used as a quote asset in Exchange for TROY/USDT swaps.	USDT behaves according to the expected ERC20 interface and remains transferable.	Tether-controlled blacklisting or other token-level restrictions can cause USDT transfers to revert and may make affected funds or operations unavailable.
USDC	External ERC20 Contract	Used as a quote asset in Exchange for TROY/USDC swaps.	USDC behaves according to the expected ERC20 interface and remains transferable.	Issuer-level controls, including blacklisting or freezing, can cause transfers to revert or restrict access to funds.
Physical Gold / Certificate Issuer	Off-chain Dependency	Underlying physical gold is represented through serialized certificates that back the TROY token.	Certificates accurately correspond to genuine, allocated physical gold.	Off-chain custody, certificate authenticity, redemption, or reconciliation failures cannot be fully enforced on-chain.

2. Entry & Exit Points Analysis (Function-Level)

2.1 Function Entry / Exit Table

Contract Name	Function	Category	
CertificatesNFT.sol	`constructor()`	**What this function can do** Initializes the implementation contract. **Main invariant(s)** Implementation remains inert; live initialization occurs through the intended deployment flow.	**What this function cannot/should not do** Should not create certificates or configure live protocol state. **Access level** Deployment-only

Contract Name	Function	Category
CertificatesNFT.sol	`initialize()`	**What this function can do** Initializes NFT name, symbol, owner, and the permanent vault binding. **What this function cannot/should not do** Cannot be executed more than once, and cannot bind a zero or non-contract vault. **Main invariant(s)** Initialization is one-shot, ownership is established correctly, and the vault authorized to mint and burn is fixed; no setter exists. **Access level** External; initializer
CertificatesNFT.sol	`setCompliance()`	**What this function can do** Configures the compliance registry. **What this function cannot/should not do** Unauthorized accounts cannot modify compliance configuration. **Main invariant(s)** Certificate compliance checks use the configured registry. **Access level** onlyOwner
CertificatesNFT.sol	`setTransfersPaused()`	**What this function can do** Pauses or resumes certificate transfers. **What this function cannot/should not do** Should not bypass required vault custody controls. **Main invariant(s)** Ordinary transfers respect the configured pause state. **Access level** onlyOwner
CertificatesNFT.sol	`infoOf()`	**What this function can do** Returns certificate information. **What this function cannot/should not do** Cannot modify certificate state. **Main invariant(s)** Returned information corresponds to the requested certificate. **Access level** Public view
CertificatesNFT.sol	`tokenIdBySerial()`	**What this function can do** Returns the token ID associated with a certificate serial number. **What this function cannot/should not do** Cannot modify serial mappings. **Main invariant(s)** Serial-to-token mapping remains consistent. **Access level** Public view

Contract Name	Function	Category	
CertificatesNFT.sol	`nextTokenId()`	**What this function can do** Returns the next certificate token ID. **Main invariant(s)** Token IDs advance monotonically.	**What this function cannot/should not do** Cannot modify the counter. **Access level** Public view
CertificatesNFT.sol	`mintTo()`	**What this function can do** Creates a certificate NFT for a specified recipient and records its certificate data. **Main invariant(s)** Only the authorized vault can increase certificate supply.	**What this function cannot/should not do** Arbitrary users cannot mint certificates. **Access level** onlyVault
CertificatesNFT.sol	`burn()`	**What this function can do** Burns a certificate NFT held by the vault when a withdrawal completes, deleting its certificate information and freeing its serial number. **Main invariant(s)** Only the authorized vault can decrease certificate supply; a burned token ID is never reissued, and a returning bar can only re-enter through a new certificate with a fresh custody attestation.	**What this function cannot/should not do** Arbitrary users cannot burn protocol certificates, and the vault cannot burn a certificate it does not hold. **Access level** onlyVault; vault custody required
CertificatesNFT.sol	`tokenURI()`	**What this function can do** Returns certificate metadata URI. **Main invariant(s)** Metadata remains associated with the correct token.	**What this function cannot/should not do** Cannot modify state. **Access level** Public view

Contract Name	Function	Category	
CertificatesNFT.sol	`supportsInterface()`	**What this function can do** Reports supported interfaces. **Main invariant(s)** ERC721/interface detection remains standards-compatible.	**What this function cannot/should not do** Cannot modify state. **Access level** Public view
ComplianceRegistry.sol	`constructor()`	**What this function can do** Initializes the implementation contract. **Main invariant(s)** Operational configuration is established through initialization.	**What this function cannot/should not do** Should not configure live compliance state on the implementation. **Access level** Deployment-only
ComplianceRegistry.sol	`initialize()`	**What this function can do** Initializes compliance roles and sanctions configuration. **Main invariant(s)** Compliance configuration has a single initialization state.	**What this function cannot/should not do** Cannot be initialized more than once. **Access level** External; initializer
ComplianceRegistry.sol	`setSanctionsOracle()`	**What this function can do** Sets the external sanctions-list contract and its enabled state. **Main invariant(s)** Sanctions checks use the configured oracle when enabled.	**What this function cannot/should not do** Unauthorized users cannot replace the sanctions source. **Access level** ADMIN_ROLE
ComplianceRegistry.sol	`setSanctionsEnabled()`	**What this function can do** Enables or disables sanctions checking. **Main invariant(s)** Sanctions enforcement follows the configured flag.	**What this function cannot/should not do** Unauthorized users cannot change the setting. **Access level** ADMIN_ROLE

Contract Name	Function	Category	
ComplianceRegistry.sol	`setSanctionsFailureMode()`	**What this function can do** Configures whether sanctions-oracle failures block operations. **Main invariant(s)** Oracle failure behavior follows the configured mode.	**What this function cannot/should not do** Unauthorized users cannot alter failure behavior. **Access level** ADMIN_ROLE
ComplianceRegistry.sol	`setAllowlistEnabled()`	**What this function can do** Enables or disables allowlist enforcement. **Main invariant(s)** Allowlist enforcement follows configuration.	**What this function cannot/should not do** Unauthorized users cannot modify the setting. **Access level** ADMIN_ROLE
ComplianceRegistry.sol	`setBlocked()`	**What this function can do** Blocks or unblocks an account. **Main invariant(s)** Blocked accounts fail applicable compliance/transfer checks.	**What this function cannot/should not do** Unauthorized users cannot modify blocked status. **Access level** COMPLIANCE_ROLE
ComplianceRegistry.sol	`setBlockedBatch()`	**What this function can do** Updates blocked status for multiple accounts. **Main invariant(s)** Each account receives the intended blocked status.	**What this function cannot/should not do** Cannot process invalid/mismatched input arrays. **Access level** COMPLIANCE_ROLE
ComplianceRegistry.sol	`setAllowedUntil()`	**What this function can do** Sets an account's allowlist expiration timestamp. **Main invariant(s)** Allowlist validity is determined from the stored expiry.	**What this function cannot/should not do** Unauthorized users cannot modify allowlist state. **Access level** COMPLIANCE_ROLE

Contract Name	Function	Category	
ComplianceRegistry.sol	`setAllowedUntilBatch()`	**What this function can do** Updates allowlist expiry for multiple accounts. **Main invariant(s)** Each account receives its corresponding expiry.	**What this function cannot/should not do** Cannot process mismatched arrays. **Access level** COMPLIANCE_ROLE
ComplianceRegistry.sol	`requireCompliant()`	**What this function can do** Validates whether an account satisfies compliance requirements. **Main invariant(s)** Non-compliant accounts cause the check to revert.	**What this function cannot/should not do** Cannot modify compliance state. **Access level** External view
ComplianceRegistry.sol	`requireTransferAllowed()`	**What this function can do** Validates transfer eligibility. **Main invariant(s)** Blocked/sanctioned/non-allowed accounts cannot pass the transfer check.	**What this function cannot/should not do** Cannot modify state. **Access level** External view
ComplianceRegistry.sol	`isCompliant()`	**What this function can do** Returns account compliance status. **Main invariant(s)** Result reflects configured compliance policy.	**What this function cannot/should not do** Cannot modify state. **Access level** Public view
ComplianceRegistry.sol	`isTransferAllowed()`	**What this function can do** Returns whether an account may participate in transfers. **Main invariant(s)** Transfer eligibility reflects block, sanctions, and allowlist rules.	**What this function cannot/should not do** Cannot modify state. **Access level** Public view

Contract Name	Function	Category	
Exchange.sol	`initialize()`	**What this function can do** Initializes supported assets, price oracles, exchange parameters (including the cross-feed price skew tolerance), roles, and compliance configuration. **Main invariant(s)** Exchange starts with valid asset, oracle, fee, limit, and price-skew configuration, with both swap directions disabled.	**What this function cannot/should not do** Cannot be executed more than once, and cannot initialize with a zero price skew tolerance or one at or above priceUpdateTimeout. **Access level** External; initializer
Exchange.sol	`exchangeStableToTroy()`	**What this function can do** Accepts USDT/USDC and transfers calculated TROY to the user. **Main invariant(s)** Output satisfies slippage protection and exchange accounting remains consistent.	**What this function cannot/should not do** Cannot execute with invalid or stale pricing, gold and stablecoin feed timestamps further apart than maxPriceSkew, insufficient liquidity, disabled exchange, or non-compliant users. **Access level** External; nonReentrant
Exchange.sol	`exchangeTroyToStable()`	**What this function can do** Accepts TROY and transfers calculated USDT/USDC to the user. **Main invariant(s)** Stablecoin output satisfies the minimum-output requirement.	**What this function cannot/should not do** Cannot exceed available stablecoin liquidity or bypass compliance/pricing checks, including the cross-feed price skew check. **Access level** External; nonReentrant

Contract Name	Function	Category	
Exchange.sol	`getQuoteStableToTroy()`	**What this function can do** Calculates a stablecoin-to-TROY quote. **Main invariant(s)** Quote reflects applicable oracle price and fee configuration, and reports unhealthy when a price is stale or the feeds are further apart than maxPriceSkew.	**What this function cannot/should not do** Cannot modify exchange state. **Access level** External view
Exchange.sol	`getQuoteTroyToStable()`	**What this function can do** Calculates a TROY-to-stablecoin quote. **Main invariant(s)** Quote reflects current pricing and fee configuration, and reports unhealthy when a price is stale or the feeds are further apart than maxPriceSkew.	**What this function cannot/should not do** Cannot modify exchange state. **Access level** External view
Exchange.sol	`setExchangeParameters()`	**What this function can do** Updates exchange fee and amount parameters. **Main invariant(s)** Fee and amount bounds remain valid.	**What this function cannot/should not do** Cannot configure values outside protocol limits. **Access level** ADMIN_ROLE
Exchange.sol	`setPriceUpdateTimeout()`	**What this function can do** Changes the accepted oracle price age. **Main invariant(s)** Stale prices are rejected according to the configured threshold, and the skew tolerance always stays below it.	**What this function cannot/should not do** Unauthorized users cannot alter it, and it cannot be set at or below the current maxPriceSkew. **Access level** ADMIN_ROLE

Contract Name	Function	Category	
Exchange.sol	`setMaxPriceSkew()`	**What this function can do** Changes the maximum timestamp difference between the gold and stablecoin price feeds. **Main invariant(s)** The cross-feed skew check is always active; swaps only execute when feed timestamps are within the configured skew.	**What this function cannot/should not do** Cannot be modified by unauthorized users, and cannot be set to zero or to a value at or above priceUpdateTimeout. **Access level** ADMIN_ROLE
Exchange.sol	`setDailyExchangeLimit()`	**What this function can do** Updates the per-user daily exchange limit. **Main invariant(s)** User exchange volume remains bounded by the configured limit.	**What this function cannot/should not do** Cannot bypass the global/user limit mechanism. **Access level** ADMIN_ROLE
Exchange.sol	`setGlobalDailyExchangeLimit()`	**What this function can do** Updates the global daily exchange limit. **Main invariant(s)** Global daily exchange volume remains bounded.	**What this function cannot/should not do** Unauthorized users cannot modify it. **Access level** ADMIN_ROLE
Exchange.sol	`setExchangeEnabled()`	**What this function can do** Enables/disables exchange directions. **Main invariant(s)** Disabled exchange directions reject swaps; enabled directions always have an active skew check.	**What this function cannot/should not do** Unauthorized users cannot change direction availability, and no direction can be enabled while the price skew configuration is invalid. **Access level** ADMIN_ROLE

Contract Name	Function	Category	
Exchange.sol	`setCompliance()`	**What this function can do** Changes the exchange compliance registry. **Main invariant(s)** Exchange operations use the configured compliance registry.	**What this function cannot/should not do** Unauthorized users cannot change it. **Access level** ADMIN_ROLE
Exchange.sol	`addStableLiquidity()`	**What this function can do** Adds USDT/USDC liquidity and updates tracked reserves. **Main invariant(s)** Tracked liquidity corresponds to the received token amount.	**What this function cannot/should not do** Unauthorized users cannot add liquidity through the privileged path. **Access level** OPERATOR_ROLE; nonReentrant
Exchange.sol	`addTroyLiquidity()`	**What this function can do** Adds TROY liquidity to the exchange. **Main invariant(s)** Tracked TROY liquidity remains consistent with actual balances.	**What this function cannot/should not do** Unauthorized users cannot perform the privileged liquidity operation. **Access level** OPERATOR_ROLE; nonReentrant
Exchange.sol	`removeTroyLiquidity()`	**What this function can do** Removes tracked TROY liquidity to a specified address. **Main invariant(s)** Protected TROY principal remains solvent.	**What this function cannot/should not do** Cannot withdraw more than available tracked liquidity. **Access level** OPERATOR_ROLE; nonReentrant
Exchange.sol	`removeStableLiquidity()`	**What this function can do** Removes tracked USDT/USDC liquidity. **Main invariant(s)** Tracked stablecoin reserves remain properly accounted for.	**What this function cannot/should not do** Cannot withdraw protected principal beyond available liquidity. **Access level** OPERATOR_ROLE; nonReentrant

Contract Name	Function	Category	
Exchange.sol	`sweepStable()`	**What this function can do** Transfers accumulated stablecoin fees to the designated destination. **Main invariant(s)** Only fee balances are sweepable.	**What this function cannot/should not do** Cannot sweep tracked principal liquidity. **Access level** OPERATOR_ROLE; nonReentrant
Exchange.sol	`rescueTroySurplus()`	**What this function can do** Recovers TROY held above tracked reserves. **Main invariant(s)** Only genuine TROY surplus is recoverable.	**What this function cannot/should not do** Cannot withdraw tracked principal. **Access level** ADMIN_ROLE; nonReentrant
Exchange.sol	`rescueStableSurplus()`	**What this function can do** Recovers stablecoin balances above tracked reserves/fees. **Main invariant(s)** Only stablecoin surplus can be rescued.	**What this function cannot/should not do** Cannot withdraw protected liquidity. **Access level** ADMIN_ROLE; nonReentrant
Exchange.sol	`pause() / unpause()`	**What this function can do** Stops or resumes protected exchange operations. **Main invariant(s)** Paused state prevents protected operations.	**What this function cannot/should not do** Unauthorized users cannot change pause state. **Access level** ADMIN_ROLE
Exchange.sol	`setEmergencyMode()`	**What this function can do** Enables or disables emergency mode. **Main invariant(s)** Emergency mode restricts affected exchange operations.	**What this function cannot/should not do** Unauthorized users cannot change emergency state. **Access level** ADMIN_ROLE

Contract Name	Function	Category	
PriceOracle.sol	`initialize()`	**What this function can do** Initializes Chainlink feeds, stale thresholds, bounds, sequencer configuration, and roles. **Main invariant(s)** Oracle starts with validated configuration.	**What this function cannot/should not do** Cannot be executed more than once. **Access level** External; initializer
PriceOracle.sol	`read()`	**What this function can do** Reads and normalizes the external asset price. **Main invariant(s)** Stale, invalid, paused, or unsafe prices are not treated as valid prices.	**What this function cannot/should not do** Cannot modify oracle configuration. **Access level** External view
PriceOracle.sol	`setPriceFeed()`	**What this function can do** Replaces the configured Chainlink price feed. **Main invariant(s)** Active price feed passes configured validation.	**What this function cannot/should not do** Cannot configure an invalid feed. **Access level** ADMIN_ROLE
PriceOracle.sol	`setSequencerUptimeFeed()`	**What this function can do** Configures L2 sequencer uptime protection and grace period. **Main invariant(s)** Sequencer downtime/grace-period conditions are respected.	**What this function cannot/should not do** Cannot be modified by unauthorized users. **Access level** ADMIN_ROLE
PriceOracle.sol	`setStaleAfter()`	**What this function can do** Updates the maximum accepted price age. **Main invariant(s)** Prices older than the threshold are invalid.	**What this function cannot/should not do** Unauthorized accounts cannot change it. **Access level** ADMIN_ROLE

Contract Name	Function	Category	
PriceOracle.sol	`setBounds()`	**What this function can do** Configures optional minimum and maximum price bounds. **Main invariant(s)** When enabled, prices remain within the configured range.	**What this function cannot/should not do** Cannot configure invalid bounds. **Access level** ADMIN_ROLE
PriceOracle.sol	`pause() / unpause()`	**What this function can do** Pauses or resumes oracle operation. **Main invariant(s)** Paused oracle data cannot be treated as valid protocol pricing.	**What this function cannot/should not do** Unauthorized users cannot change oracle pause state. **Access level** ADMIN_ROLE
PriceOracle.sol	`latestRoundDataRaw()`	**What this function can do** Returns raw Chainlink round data. **Main invariant(s)** Raw oracle data is exposed without changing configuration.	**What this function cannot/should not do** Cannot modify state. **Access level** External view
PriceOracle.sol	`feedMetadata()`	**What this function can do** Returns configured feed metadata. **Main invariant(s)** Metadata corresponds to the configured feed.	**What this function cannot/should not do** Cannot modify state. **Access level** External view
TroyToken.sol	`constructor()`	**What this function can do** Initializes the TROY token, owner, and the immutable vault address. **Main invariant(s)** Mint/burn authority is permanently restricted to the vault set at construction; no setter exists.	**What this function cannot/should not do** Cannot bind a zero or non-contract vault, and cannot provide arbitrary post-deployment minting authority. **Access level** Deployment-only

Contract Name	Function	Category	
TroyToken.sol	`setCompliance()`	**What this function can do** Configures the compliance registry. **Main invariant(s)** Transfers use the configured compliance policy.	**What this function cannot/should not do** Unauthorized users cannot modify it. **Access level** onlyOwner
TroyToken.sol	`setTransfersPaused()`	**What this function can do** Pauses or resumes ordinary TROY transfers. **Main invariant(s)** Ordinary transfers respect pause state.	**What this function cannot/should not do** Should not bypass vault custody controls. **Access level** onlyOwner
TroyToken.sol	`mint()`	**What this function can do** Mints TROY against an authorized vault reference. **Main invariant(s)** Only the vault can increase TROY total supply.	**What this function cannot/should not do** Arbitrary accounts cannot increase supply. **Access level** onlyVault
TroyToken.sol	`burn()`	**What this function can do** Burns TROY against an authorized vault reference. **Main invariant(s)** Only the vault can decrease TROY supply.	**What this function cannot/should not do** Arbitrary accounts cannot decrease supply through this function. **Access level** onlyVault

Contract Name	Function	Category	
VaultManager.sol	`initialize()`	**What this function can do** Initializes vault dependencies, roles, certificate NFT, TROY token, compliance, and limits. In the standard deployment it is called by VaultSystemDeployer in the same transaction that creates the vault proxy and binds both assets to it.	**What this function cannot/should not do** Cannot be executed more than once, and cannot accept colliding role addresses.
		Main invariant(s) Vault configuration is established exactly once; the TROY and certificate addresses cannot change afterwards.	**Access level** External; initializer
VaultManager.sol	`requestCertificate()`	**What this function can do** Creates a certificate issuance request.	**What this function cannot/should not do** Cannot create invalid requests or bypass compliance/weight limits.
		Main invariant(s) Each request records its requester and requested gold weight.	**Access level** External; whenNotPaused, nonReentrant
VaultManager.sol	`approveCertificate()`	**What this function can do** Approves a certificate request and mints the corresponding certificate NFT.	**What this function cannot/should not do** Cannot approve invalid or already-finalized requests.
		Main invariant(s) Minted certificate data matches the approved request.	**Access level** CERTIFIER_ROLE; nonReentrant
VaultManager.sol	`rejectCertificate()`	**What this function can do** Rejects a pending certificate request.	**What this function cannot/should not do** Cannot reject a finalized request.
		Main invariant(s) Rejected requests cannot result in certificate minting.	**Access level** CERTIFIER_ROLE; nonReentrant

Contract Name	Function	Category	
VaultManager.sol	`cancelCertificate()`	**What this function can do** Allows the requester to cancel a pending certificate request. **Main invariant(s)** Cancelled requests cannot subsequently mint certificates.	**What this function cannot/should not do** Cannot cancel another user's or finalized request. **Access level** Requester; nonReentrant
VaultManager.sol	`requestDeposit()`	**What this function can do** Escrows certificates and creates a deposit request. **Main invariant(s)** Each escrowed certificate is associated with the pending deposit request.	**What this function cannot/should not do** Cannot accept duplicate, unavailable, or improperly owned certificates. **Access level** External; whenNotPaused, nonReentrant
VaultManager.sol	`approveDeposit()`	**What this function can do** Approves a deposit and mints the corresponding TROY. **Main invariant(s)** TROY minted corresponds to the approved backing weight.	**What this function cannot/should not do** Cannot approve invalid requests or mismatched certificate lists. **Access level** OPERATOR_ROLE; nonReentrant
VaultManager.sol	`rejectDeposit()`	**What this function can do** Rejects a deposit and returns escrowed certificates. **Main invariant(s)** No TROY is minted for a rejected deposit.	**What this function cannot/should not do** Cannot reject finalized requests. **Access level** OPERATOR_ROLE; nonReentrant
VaultManager.sol	`cancelDeposit()`	**What this function can do** Allows a requester to cancel a pending deposit. **Main invariant(s)** Escrowed certificates are released without minting TROY.	**What this function cannot/should not do** Cannot cancel another user's or finalized request. **Access level** Requester; nonReentrant

Contract Name	Function	Category	
VaultManager.sol	`requestWithdraw()`	**What this function can do** Escrows TROY and creates a withdrawal request. **Main invariant(s)** Escrowed TROY remains associated with the pending withdrawal.	**What this function cannot/should not do** Cannot accept invalid amounts or non-compliant users. **Access level** External; whenNotPaused, nonReentrant
VaultManager.sol	`approveWithdraw()`	**What this function can do** Allocates backing certificates to an approved withdrawal. **Main invariant(s)** Each certificate can only be allocated to the appropriate withdrawal state.	**What this function cannot/should not do** Cannot allocate unavailable or already-reserved certificates. **Access level** OPERATOR_ROLE; nonReentrant
VaultManager.sol	`completeWithdraw()`	**What this function can do** Completes an approved withdrawal, burns the reserved certificate NFTs and the matching TROY, reduces recorded backing, and refunds any unused TROY escrow. Callable after the cancellation deadline and while paused. **Main invariant(s)** TROY burned corresponds to the fulfilled backing value, and a redeemed certificate can never back TROY again. Completion must be recorded on-chain before physical gold is released.	**What this function cannot/should not do** Cannot complete invalid or unauthorized withdrawal requests, reuse a fulfillment reference, or return a redeemed certificate to the user. **Access level** CUSTODIAN_ROLE; nonReentrant

Contract Name	Function	Category	
VaultManager.sol	`revokeWithdrawApproval()`	**What this function can do** Revokes a withdrawal approval, releases allocated certificates, and returns the request to Pending. **Main invariant(s)** Released certificates become available for subsequent valid allocation; the seven-day cancellation deadline stays anchored to the first approval, so revoking and re-approving cannot extend the user's escrow lock.	**What this function cannot/should not do** Cannot revoke invalid or non-approved requests, and does not reset the original approval timestamp. **Access level** OPERATOR_ROLE; nonReentrant
VaultManager.sol	`rejectWithdraw()`	**What this function can do** Rejects a withdrawal request and refunds the user's escrowed TROY. **Main invariant(s)** Rejected withdrawals do not burn the user's TROY.	**What this function cannot/should not do** Cannot reject finalized requests. **Access level** OPERATOR_ROLE; nonReentrant
VaultManager.sol	`cancelWithdraw()`	**What this function can do** Allows a requester to cancel an eligible withdrawal request. **Main invariant(s)** Cancellation releases reserved resources and refunds escrow appropriately.	**What this function cannot/should not do** Cannot cancel another user's or finalized request. **Access level** Requester; nonReentrant
VaultManager.sol	`recoverDeposit()`	**What this function can do** Recovers an affected deposit for a blocked/non-compliant user through the administrative recovery path. **Main invariant(s)** Recovered certificate state and protocol accounting are finalized consistently.	**What this function cannot/should not do** Cannot recover invalid or ineligible requests. **Access level** ADMIN_ROLE; nonReentrant

Contract Name	Function	Category	
VaultManager.sol	`recoverWithdrawal()`	**What this function can do** Recovers an affected withdrawal for a blocked/non-compliant user. **Main invariant(s)** Locked withdrawal state has a deterministic recovery path.	**What this function cannot/should not do** Cannot recover invalid or finalized requests. **Access level** ADMIN_ROLE; nonReentrant
VaultManager.sol	`pause() / unpause()`	**What this function can do** Pauses or resumes vault operations. **Main invariant(s)** New lifecycle operations respect the pause state.	**What this function cannot/should not do** Unauthorized users cannot modify pause state. **Access level** ADMIN_ROLE
VaultManager.sol	`setCompliance()`	**What this function can do** Changes the compliance registry used by vault operations. **Main invariant(s)** Compliance-sensitive operations use the configured registry.	**What this function cannot/should not do** Unauthorized users cannot change compliance configuration. **Access level** ADMIN_ROLE
VaultManager.sol	`setDailyMintLimit()`	**What this function can do** Updates the daily TROY mint limit. **Main invariant(s)** Daily TROY issuance remains within the configured limit.	**What this function cannot/should not do** Cannot bypass the issuance circuit breaker. **Access level** ADMIN_ROLE
VaultManager.sol	`setMaxCertificateWeight()`	**What this function can do** Changes the maximum certificate weight. **Main invariant(s)** Certificate issuance remains bounded by the configured maximum.	**What this function cannot/should not do** Unauthorized users cannot change the limit. **Access level** ADMIN_ROLE

Contract Name	Function	Category	
VaultManager.sol	`rescueERC20()`	**What this function can do** Recovers eligible ERC20 tokens accidentally held by the vault. **Main invariant(s)** Protocol-owned backing remains protected.	**What this function cannot/should not do** Cannot withdraw protected TROY backing required for solvency. **Access level** ADMIN_ROLE; nonReentrant
VaultManager.sol	`rescueERC721()`	**What this function can do** Recovers eligible ERC721 tokens accidentally held by the vault. **Main invariant(s)** Backing certificates remain under intended vault custody.	**What this function cannot/should not do** Cannot withdraw protocol backing certificates through the rescue path. **Access level** ADMIN_ROLE; nonReentrant
VaultManager.sol	`getCertificateRequest()`	**What this function can do** Returns certificate-request information. **Main invariant(s)** Returned data reflects stored request state.	**What this function cannot/should not do** Cannot modify request state. **Access level** External view
VaultManager.sol	`getDepositRequest()`	**What this function can do** Returns deposit-request information. **Main invariant(s)** Returned data reflects current deposit lifecycle state.	**What this function cannot/should not do** Cannot modify state. **Access level** External view
VaultManager.sol	`getWithdrawalRequest()`	**What this function can do** Returns withdrawal-request information. **Main invariant(s)** Returned information reflects the current withdrawal state.	**What this function cannot/should not do** Cannot modify state. **Access level** External view

Contract Name	Function	Category	
VaultManager.sol	`onChainBackingMatchesSupply()`	**What this function can do** Checks whether on-chain backing corresponds to TROY supply. **Main invariant(s)** Backing/supply consistency is correctly reported.	**What this function cannot/should not do** Cannot modify backing or supply. **Access level** External view
VaultManager.sol	`onERC721Received()`	**What this function can do** Accepts ERC721 safe transfers to the vault. **Main invariant(s)** Returns the ERC721 receiver selector and custody accounting remains lifecycle-controlled.	**What this function cannot/should not do** Receiving an NFT alone should not incorrectly finalize a certificate lifecycle. **Access level** Public
VaultSystemDeployer.sol	`constructor()`	**What this function can do** Deploys the VaultManager proxy, a TroyToken and a CertificatesNFT proxy bound to that vault, then initializes the vault with both asset addresses, all in one transaction. **Main invariant(s)** The vault and both assets are bound to each other at deployment.	**What this function cannot/should not do** Cannot be used with pre-set asset addresses, cannot leave the vault proxy uninitialized between transactions, and retains no authority after deployment. **Access level** Deployment-only

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
VaultManager.sol	ERC20 / ERC721	TROY tokens held in withdrawal escrow and physical-gold certificate NFTs held as backing/escrow

Contract	Asset Type	Custodied Assets
Exchange.sol	ERC20	TROY, USDT, and USDC reserves, including accumulated exchange fees
CertificatesNFT.sol	ERC721	Certificate NFTs representing serialized physical gold certificates

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
`VaultManager.sol`	`requestDeposit()`	Certificate NFTs	-	Public	Certificate ownership, duplicate-token, escrow, and compliance checks
`VaultManager.sol`	`approveDeposit()`	Escrowed Certificate NFTs	TROY	Operator	Gold-weight-to-TROY accounting and unauthorized minting risk
`VaultManager.sol`	`rejectDeposit()`	-	Certificate NFTs	Operator	Incorrect request-state handling may lock or misroute certificates
`VaultManager.sol`	`cancelDeposit()`	-	Certificate NFTs	Requester	Unauthorized cancellation and escrow-release risk
`VaultManager.sol`	`requestWithdraw()`	TROY	-	Public	TROY escrow, compliance, amount-validation, and locking risk
`VaultManager.sol`	`approveWithdraw()`	-	Certificate NFTs allocated	Operator	Certificate allocation and double-allocation risk

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
VaultManager.sol	completeWithdraw()	Escrowed TROY, reserved certificate NFTs	Unused TROY escrow refunded to user (redeemed TROY and certificates are burned; physical gold released off-chain)	Custodian	Burn/backing accounting and fulfillment risk; completion must be recorded on-chain before the physical bar is released
VaultManager.sol	rejectWithdraw()	-	TROY	Operator	Incorrect refund or request-state transition risk
VaultManager.sol	cancelWithdraw()	-	TROY	Requester	Escrow-release and authorization risk
VaultManager.sol	recoverDeposit()	Escrowed Certificate NFTs	Certificate NFTs	Admin	Recovery-address and blocked-user recovery risk
VaultManager.sol	recoverWithdrawal()	Escrowed TROY / withdrawal state	Recovered assets	Admin	Recovery authorization and accounting risk
Exchange.sol	exchangeStableToTroy()	USDT / USDC	TROY	Public	Oracle manipulation, stale pricing, slippage, liquidity, and stablecoin blacklist risk
Exchange.sol	exchangeTroyToStable()	TROY	USDT / USDC	Public	Oracle manipulation, stale pricing, slippage, liquidity, and stablecoin blacklist risk
Exchange.sol	addStableLiquidity()	USDT / USDC	-	Operator	Incorrect reserve accounting and non-standard token behavior

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Exchange.sol	addTroyLiquidity()	TROY	-	Operator	Reserve accounting and privileged liquidity risk
Exchange.sol	removeStableLiquidity()	-	USDT / USDC	Operator	Principal withdrawal and reserve solvency risk
Exchange.sol	removeTroyLiquidity()	-	TROY	Operator	Principal withdrawal and reserve solvency risk
Exchange.sol	sweepStable()	-	USDT / USDC fees	Operator	Fee-accounting and stablecoin transfer/blacklist risk
Exchange.sol	rescueStableSurplus()	-	USDT / USDC surplus	Admin	Incorrect surplus calculation could expose protected reserves
Exchange.sol	rescueTroySurplus()	-	TROY surplus	Admin	Incorrect surplus calculation could expose protected reserves
CertificatesNFT.sol	mintTo()	-	Certificate NFT	Vault	Unauthorized minting and certificate-data integrity risk
CertificatesNFT.sol	burn()	Certificate NFT	-	Vault (via completeWithdraw())	Unauthorized burning and backing-accounting risk; burning frees the serial number, so a returning bar must be re-certified with a fresh custody attestation

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TroyToken.sol	mint()	-	TROY	Vault	Unauthorized inflation and backing mismatch risk
TroyToken.sol	burn()	TROY	-	Vault	Unauthorized burning and supply/accounting mismatch risk

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Aum Money. We performed our audit according to the procedure described above.

5 issues were identified in Aum Money out of which 2 issues were acknowledged and 3 issues were fixed in subsequent Fix review.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

October 2026

For



QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com